

Python Assignment 08

1. Explain the steps to create and use a user-defined module.

Step 1: Create a Module

File: A8Q1.py

```
def Display():  
    print("Jai Ganesh...")  
  
PI = 3.14159
```

Step 2: Import and Use in Another File.

File: A8Q1(1).py

```
import A8Q1  
  
A8Q1.Display()  
print("Value of PI:", A8Q1.PI)
```

Output:

Jai Ganesh...

Value of pi: 3.14159

2. Why is indentation mandatory in Python while it is optional in languages like C, C++ or Java?

In C, C++, Java, the block boundaries are marked explicitly using { }: that braces do the job.

```
if (x > 0) {  
    printf("positive");  
}
```

In Python,

```
if 10 > 5:  
    print("10 is greater than 5")  
    print("Inside the if block")  
  
print("Outside the if block")
```

OUTPUT :

```
10 is greater than 5  
Inside the if block  
Outside the if block
```

See the another on python example

```
age = 20  
if age >= 18:
```

```
print("Eligible")
```

```
    print("Hello")
```

OUTPUT :

```
print("Hello")
```

```
IndentationError: unexpected indent
```

3. What is an IndentationError? When does Python raise this error?

An **IndentationError** is a syntax error raised by Python when the indentation of code is incorrect. Python raises this error when a required block is not indented, when there is unexpected indentation, when indentation levels are inconsistent, or when tabs and spaces are mixed improperly. Since Python uses indentation to define code blocks, correct indentation is mandatory.

```
def main():
```

```
    age = 20
```

```
    if age >= 18:
```

```
        print("Eligible")          # using by Tab
```

```
        print("Can vote")         #using by Spaces
```

```
        print("Eligible to vote")
```

```
if __name__ == "__main__":
```

```
    main()
```

```
#TabError:inconsistent use of tabs and spaces in indentation
```

4. What are the different ways to import a module? Which one is recommended and why?

Answer :

Python provides several ways to import a module:

```
#Module 1
```

```
import math
```

```
print(math.sqrt(25))
```

```
print(math.pi)
```

```
#Module 2
```

```
import math as m
```

```
print(m.sqrt(16))
```

```
print(m.pi)

#Module 3
from math import sqrt

print(sqrt(49))

#Module 4
from math import sqrt, pi

print(sqrt(64))
print(pi)

#Module 5
from math import *

print(sqrt(81))
print(pi)
```

The **recommended** method is **import module** because it clearly identifies the source of functions and variables, avoids namespace conflicts, and improves code readability and maintainability. The `from module import *` form is generally discouraged because it imports all public names into the current namespace, increasing the risk of name collisions.

5. How does Python internally identify the start and end of a code block using indentation?

Answer :

Python identifies the start and end of a code block by comparing the **indentation level** of consecutive lines. After a statement ending with a colon (:), an increase in indentation starts a new block. Statements with the same indentation belong to the same block, while a decrease in indentation marks the end of the current block. Thus, Python uses indentation instead of braces ({}) to define code blocks.

```
def main():
    x = 10

    if x > 0:
        print("Positive")

        if x > 5:
            print("Greater than 5")

        print("Done")

    print("End")
```

```
if __name__ == "__main__":  
    main()
```

Notice:

- **4 spaces** → Start of the if block.
- **8 spaces** → Start of the nested if block.
- Returning to **4 spaces** → End of the nested block.
- Returning to **0 spaces** → End of the outer block.

6. Why is from module import * discouraged in professional code?

Answer:

from module import * is discouraged in professional Python code because it imports all public names from a module into the current namespace. This can cause **namespace pollution**, **name conflicts**, and make code **harder to read, maintain, and debug**. The recommended approach is to use import module or from module import specific_name, which clearly indicates the origin of imported objects and improves code readability.

```
from math import *  
from math import * #Name Conflicts  
from cmath import * #sqrt() library define in math and cmath module  
  
def main():  
    #1 NameSpace Pollution  
    print(sqrt(25))  
    print(pi)  
    #After After this import, names like: sqrt, sin, cos, tan, log, pi, e  
  
    #2 Name Conflicts  
    print(sqrt(16)) #may overwrite the earlier one, making the code  
confusing.  
  
if __name__ == "__main__":  
    main()
```

7. How does modular programming improve code reusability, testing, and maintenance?

Answer :

Modular programming divides a program into small, independent modules, each performing a specific task. It improves code reusability by allowing the same module to be imported and used in multiple programs. It improves testing because each module can be tested independently, making bugs easier to identify and fix. It improves maintenance because changes are usually limited to a single module without affecting the rest of the program, resulting in more organized, readable, and maintainable code.

Main.py

```
#Author : codingwithamar@gmail.com
```

```
#Description : 'How does modular programming improve code reusability, testing, and maintenance?'
```

```
import account
import loan
import customer
import database

def main():
    database.connect()
    balance = 5000
    balance = account.deposit(balance, 2000)
    print(balance)
    print(loan.calculate_emi(100000, 0.08))
    customer.customer_info()

if __name__ == "__main__":
    main()
```

database.py

```
def connect():
    print("Database Connected")
```

account.py

```
def deposit(balance, amount):
    return balance + amount

def withdraw(balance, amount):
    return balance - amount
```

loan.py

```
def calculate_emi(principal, rate):
    return principal * rate
```

customer.py

```
def customer_info():
    print("Customer Information")
```

OUTPUT :

```
Database Connected
7000
8000.0
Customer Information
```

A **Bank Management System** can be divided into modules such as account.py, loan.py, customer.py, and database.py. The main.py file imports these modules. This improves **code reusability** because modules like account.py can be reused in other applications such as an ATM

system. It improves **testing** because each module can be tested independently. It improves **maintenance** because changes made to one module, such as `loan.py`, do not affect the other modules. As a result, the code becomes more organized, reusable, easier to test, and easier to maintain.

8. What happens internally when Python executes an `import module_name` statement?

Answer :

When Python executes `import module_name`, it first checks whether the module is already loaded in `sys.modules`. If found, it reuses the cached module. Otherwise, Python searches for the module, loads it into memory, creates a module object, executes its top-level code once, stores the module in `sys.modules`, and binds the module name to the current namespace. This caching mechanism improves performance by avoiding repeated loading and execution of the same module.

`calculator.py`

```
print("Loading calculator module...")

PI = 3.14159

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

print("Calculator module loaded successfully.")
```

`main.py`

```
import calculator

def main():
    print("Program Started")
    print("Import Completed")

    print(calculator.add(10, 20))
    print(calculator.PI)

    print("Program Finished")

if __name__ == "__main__":
    main()
```

OUTPUT:

```
Loading calculator module...
Calculator module loaded successfully.
Program Started
Import Completed
```

30
3.14159
Program Finished

9. What happens if a module contains print statements outside any function?

Answer :

If a module contains print() statements outside any function or class, those statements are **executed automatically when the module is imported for the first time** because Python executes all top-level statements during module initialization. The module is then cached in sys.modules, so subsequent imports do not execute those statements again. To prevent unwanted execution during import, such code should be placed inside an if __name__ == "__main__": block

Main.py

```
#Author : codingwithamar@gmail.com
#Description : 'What happens if a module contains print statements outside
any function?'

print("Program Started")

import calculator

print("Import Finished")

result = calculator.add(10, 20)

print("Result =", result)

print("Program End")
```

calculator.py

```
print("Step 1: Calculator module is loading...")

PI = 3.14159

print("Step 2: PI variable created.")

def add(a, b):
    return a + b

print("Step 3: add() function created.")

def subtract(a, b):
    return a - b

print("Step 4: Calculator module loaded successfully.")
```

OUTPUT :

Program Started

Step 1: Calculator module is loading...

Step 2: PI variable created.

Step 3: add() function created.

Step 4: Calculator module loaded successfully.

Import Finished

Result = 30

Program End

10. Explain why Python modules are considered the foundation of scalable software design.

Answer :

Python modules are considered the **foundation of scalable software design** because they divide large applications into **small, independent, and reusable units**. Each module performs a specific task and can be imported into other programs, improving **code reusability**. Modules also make software easier to **test, debug, and maintain**, since changes are usually limited to a single module. They enable multiple developers to work on different parts of a project simultaneously and allow software to grow in size without becoming difficult to manage.

Main.py

```
#Author : codingwithamar@gmail.com
#Description : 'How does modular programming improve code reusability,
testing, and maintenance?'

import account
import loan
import customer
import database

def main():
    database.connect()
    balance = 5000
    balance = account.deposit(balance, 2000)
    print(balance)
    print(loan.calculate_emi(100000, 0.08))
    customer.customer_info()

if __name__ == "__main__":
    main()
```

database.py

```
def connect():
    print("Database Connected")
```

account.py

```
def deposit(balance, amount):
```



```
    return balance + amount
```

```
def withdraw(balance, amount):  
    return balance - amount
```

loan.py

```
def calculate_emi(principal, rate):  
    return principal * rate
```

customer.py

```
def customer_info():  
    print("Customer Information")
```

OUTPUT :

Database Connected

7000

8000.0

Customer Information

A **Bank Management System** can be divided into modules such as account.py, loan.py, customer.py, and database.py. The main.py file imports these modules. This improves **code reusability** because modules like account.py can be reused in other applications such as an ATM system. It improves **testing** because each module can be tested independently. It improves **maintenance** because changes made to one module, such as loan.py, do not affect the other modules. As a result, the code becomes more organized, reusable, easier to test, and easier to maintain.